

INF1163 — Conception détaillée & modélisation d'une application

Travail pratique 2 — Stratégie & mise en œuvre des tests

Équipe de 2 ou 3

Remise d'un **fichier .zip** par courriel au plus tard le **14 décembre 2025**

Langages autorisés : **Java, C#, C++, Python** (OO)

1) Contexte & objectifs

Dans la continuité du TP1, vous devez concevoir et implémenter une solution de tests couvrant les niveaux : unitaire, intégration, système (end-to-end), acceptation. On attend:

- Une pyramide de tests (majorité de tests unitaires, un peu d'intégration, très peu d'E2E) et un focus risque/valeur (pas de doublons, pas de tests de bibliothèques tierces). L'approche "pyramide" est la référence ici. martinfowler.com
 - Des tests orientés exigences du TP1 : zoom, translation, vues multiples, Undo/Redo (sur la bonne vue), copier/coller des paramètres, persistance Save/Load fidèle.
 - Un rapport automatisé (résultats, couverture).
-

2) Portée & priorités (4 semaines : allez à l'essentiel)

- Unitaires (priorité élevée, Modèle & Commandes) : invariants et transitions d'état (échelle > 0, translations aux bornes de l'image, Observer émet les notifications pertinentes, Command/Undo/Redo idempotents).
- Intégration (priorité moyenne) : contrôleur↔modèle (Command), persistance Save→Load = identité (même perspective reconstruite), copier/coller entre vues (Strategy/Null).
- Système/E2E (priorité basse, échantillon critique) : 2-3 parcours démontrant "de bout en bout" (ex. menu et geste souris déclenchent la même commande). Gardez l'E2E parcimonieux (flaky/chaud à maintenir). martinfowler.com
- Acceptation (BDD/Gherkin) : scénarios lisibles métiers reliés aux exigences du TP1. [Cucumber](#)

3) Exigences minimales par niveau

- **Unitaires (≈ 70 % des cas) :**
 - **Modèle :** Image, Perspective, historiques d'Undo/Redo, "memento/snapshot" si présent.
 - **Commandes :** chaque commande (do/undo/redo), copier échelle/translation/les deux/rien.
 - **Notifications :** Observer (sujets découplés : image vs perspective).
- **Intégration (≈ 20 %) :**
 - **Contrôleurs interchangeables** ↔ mêmes effets sur le modèle (menus vs souris).
 - **Persistance :** Save→Load restaure exactement l'état (plusieurs perspectives nommées).
- **Système/E2E (≈ 10 %) :**
 - **Parcours critiques :** (a) Undo agit seulement sur la vue active ; (b) Copier sur vue A / Coller sur vue B ; (c) Redémarrage + Load reproduit l'affichage attendu.
 - **Restez minimalistes :** quelques scénarios robustes > suite massive fragile. martinfowler.com

4) Architecture & pratiques de test (directives)

- **Organisation :** test/unit, test/integration, test/e2e, test/acceptance + test/resources (images contrôlées).
- **Doubles de test :** préférez mocks/stubs pour les vues/IO (ne testez pas les pixels ; testez l'état du modèle et les événements). Pour Java : Mockito. [Mockito+1](#)
- **Données & déterminisme :** fichiers temporaires, seeds fixes, pas de dépendances réseau.
- **Styles :** AAA (Arrange-Act-Assert), noms explicites (shouldUndoOnlyActiveView_whenUndoOnA).

- **Qualité :**
 - **Couverture :** cibles raisonnables (guides) → Modèle/Commandes ≥ 80 %, Contrôleurs ≥ 70 %, Vues : pragmatique. Outil (Java) : JaCoCo. [JaCoCo+1](#)
-

5) Outils (à titre indicatif, choisissez selon votre stack)

- **Java :** *JUnit 5, Mockito, AssertJ (optionnel), JaCoCo, PIT* ; build Maven/Gradle. [PIT Mutation Testing+3JUnit+3Mockito+3](#)
 - **Python :** *pytest, pytest-mock, coverage.py, Hypothesis* (property-based). [pytest+1](#)
 - **C# :** *xUnit.net, Moq, Coverlet* (au choix). [xUnit+1](#)
 - **C++ :** *GoogleTest/GoogleMock, lcov/llvm-cov*. [Google GitHub](#)
 - **BDD/Acceptation :** *Cucumber* (Gherkin). [Cucumber](#)
-

6) Livrables (dans INF1163-TP2-ÉquipeXX.zip)

A) Plan de test (PDF, ≤ 10 pages) : contexte/projet, risques & priorités, stratégie pyramidale, matrice de traçabilité (exigences TP1 ↔ scénarios ↔ tests), critères d'entrée/sortie, dette connue. (*Vous pouvez réutiliser un gabarit IEEE 829 allégé.*)

[CSUN ECS](#)

B) Code de tests (+ fixtures, doubles, données).

C) Rapports d'exécution : résultats (JUnit XML/pytest), couverture (HTML), mutation (si appliqué), logs.

D) Courte capsule (≤ 20 s) montrant : exécution CI verte + ouverture d'un rapport de couverture.

7) Indications & garde-fous (contre les tests inutiles)

- Testez vos règles métier (invariants, états, contrats), pas la lib GUI/JSON.
- Un test = une idée. Évitez les assertions difficiles à diagnostiquer.
- Préférez tests d'intégration ciblés à des E2E coûteux/instables. [martinfowler.com](#)

- Faites des assertions robustes (sur l'état/événements), pas sur des détails d'implémentation (ex. ordre d'appels non contractuel).
 - Persistance : tests d'idempotence $\text{Load}(\text{Save}(p)) == p$ et de compatibilité (plusieurs perspectives).
-

8) Grille d'évaluation (100 pts)

- Stratégie & Plan (pyramide, risques, traçabilité) 20
 - Qualité des tests unitaires (pertinence, lisibilité, doubles) 20
 - Tests d'intégration (Command↔Modèle, persistance) 20
 - Système & Acceptation (parcours critiques, Gherkin clair) 15
 - Rapports (couverture, résultats) 15
 - Discipline (structure, données de test, README exécution) 10
-

9) Lectures & gabarits (utiles, non obligatoires)

- Pyramide de tests : explications, bonnes pratiques. martinfowler.com
 - JUnit 5 – User Guide (annotations, tests paramétrés, tags). [JUnit](#)
 - pytest – Documentation (fixtures, paramétrage, mocks). [pytest](#)
 - Mockito (mocks pour Java, JUnit Jupiter). [Mockito](#)
 - JaCoCo (couverture) & Gradle JaCoCo plugin. [JaCoCo+1](#)
 - PIT (mutation testing en Java). [PIT Mutation Testing](#)
 - Hypothesis (property-based en Python). [Hypothesis](#)
 - Cucumber / Gherkin (réf. + localisation FR). [Cucumber+1](#)
 - CI GitHub Actions (Java/Maven) : workflow starter. [GitHub Docs](#)
 - Gabarit “Test Plan” (IEEE 829, condensé) : modèle universitaire simple. [CSUN ECS](#)
-

10) Checklist (à ne pas remettre)

- **Pyramide respectée (unit > intégration > E2E) + focus risque/valeur**
 - **Matrice de traçabilité TP1 → scénarios → tests**
 - **Undo/Redo testé sur la bonne vue + copier/coller inter-vues**
 - **Persistance Save→Load prouvée (après redémarrage)**
 - **Rapports couverture et résultats attachés en annexe**
 - **Plan de test (≤ 10 p.) concis, exécutable, sans bla-bla**
-

Conseils d'alignement (3 semaines)

Semaine 1 : plan de test + squelettes unitaires sur le Modèle/Commandes.

Semaine 2 : intégration (Command↔Modèle, persistance) + 1er E2E.

Semaine 3 : acceptation (Gherkin), couverture, nettoyage.

Livrez compact, pertinent, démontrable.